



Seva Mandal Education Society's
Smt. Kamalaben Gambhirchand Shah Department Of Computer Applications
Under
DR. B.M.N COLLEGE OF HOME SCIENCE (Empowered Autonomous Status)

MEMORY GAME USING C LANGUAGE

Authors & Affiliation:

Disha Thakur
SYBCA, Roll No 164
Department of BCA,
Dr.BMN College of Home Science, Mumbai, India
Email: thakurdisha9906@gmail.com

ACKNOWLEDGEMENT

I would like to thank my college for giving me the opportunity to work on this project as a part of my academic curriculum. I appreciate the EECH Committee for assigning this project, which helped me to improve my technical knowledge, strengthen my programming skills, and gain hands-on experience.

I am grateful to my respected teachers for their constant guidance, encouragement, and valuable feedback throughout this process. Lastly, I want to acknowledge the academic environment provided by my institution, which has inspired me to explore new ideas and apply what I've learned in practical ways. This project has played an important role in my learning journey and has given a valuable chance to apply what I learned in theory to a real, interactive program and strengthened my technical and analytical skills.



Seva Mandal Education Society's

Smt. Kamalaben Gambharchand Shah Department Of Computer Applications

Under

DR. B.M.N COLLEGE OF HOME SCIENCE (Empowered Autonomous Status)

NAAC Re-Accredited 'A+' Grade with CGPA 3.69/4

UGC Status: College with Potential for Excellence

'Best College Award 2016-17 & 2024-2025' adjudged by
S.N.D.T Women's University 338, R.A.Kidwai Road Matunga,
Mumbai

CERTIFICATE

This is to certify that Ms. Disha Thakur completed the project titled
"Memory Game Using C Language" satisfactorily, and submitted the project report as per the
guidelines of S.N.D.T. Women's University, Mumbai.

EECH In-charge: Ms. Neetu Singhi

Date:

Program Coordinator: Mr. Shahjahan Khan

Date:

Members: Ms. Sukanya Vyas

Date:

College Seal:

Members: Ms. Aditi Yadav

Date:

TABLE OF CONTENTS

SR.NO	TOPIC	PAGE
I	INTRODUCTION	5
II	LITERATURE REVIEW/RELATED WORK	6
III	SYSTEM REQUIREMENTS	7
IV	METHODOLOGY	8-9
V	IMPLEMENTATION	10-18
VI	TESTING	19
VII	CONCLUSION AND FUTURE SCOPE	20

I. INTRODUCTION

▪ BACKGROUND

Memory games are simple logic-based puzzles designed to improve focus and concentration. Typically, they are played using cards placed face down, where the player flips two cards at a time to find matching pairs. Based on this idea, the project creates a basic card-matching memory game using the C programming language in the Turbo C environment. The objective of the project is to show how basic programming concepts such as arrays, loops, conditions, and randomization can be combined with simple graphics to create an interactive program.

▪ PURPOSE AND SCOPE

The main purpose of this project is to create an interactive game that allows players to have fun while improving their focus and memory. The project includes a 4x4 card grid, where players move with simple keyboard controls such as the arrow keys. The game uses pair matching logic, which reveals cards and flip them back if they don't match. Upon successfully matching all pairs, the game shows a victory message, indicating the player has successfully completed the game.

▪ IMPORTANCE AND RELEVANCE

This project is important because it shows how fundamental C programming concepts can be applied by developing an interactive game. It is relevant in an academic setting as it deepens understanding of arrays, loops, conditional logic, and graphics programming. Additionally, it provides a simple yet effective way to combine theoretical knowledge with practical application.

II. LITERATURE REVIEW/RELATED WORK

Memory games, often called concentration or matching games, come in both physical card form and digital versions. Online and mobile apps offer improved options with images, animations, and scoring systems. However, these demand modern programming tools and platforms. In contrast, this project creates a simplified version of the memory game using C language and Turbo C graphics. This approach is suitable for academic learning and shows basic programming concepts.

III. SYSTEM REQUIREMENTS

▪ HARDWARE REQUIREMENTS

The game runs on a standard PC with at least 2 GB of RAM and a VGA/SVGA compatible display. This setup ensures the graphics used in Turbo C environment are rendered properly.

▪ SOFTWARE REQUIREMENTS

The software requirements for running the game are as follows:

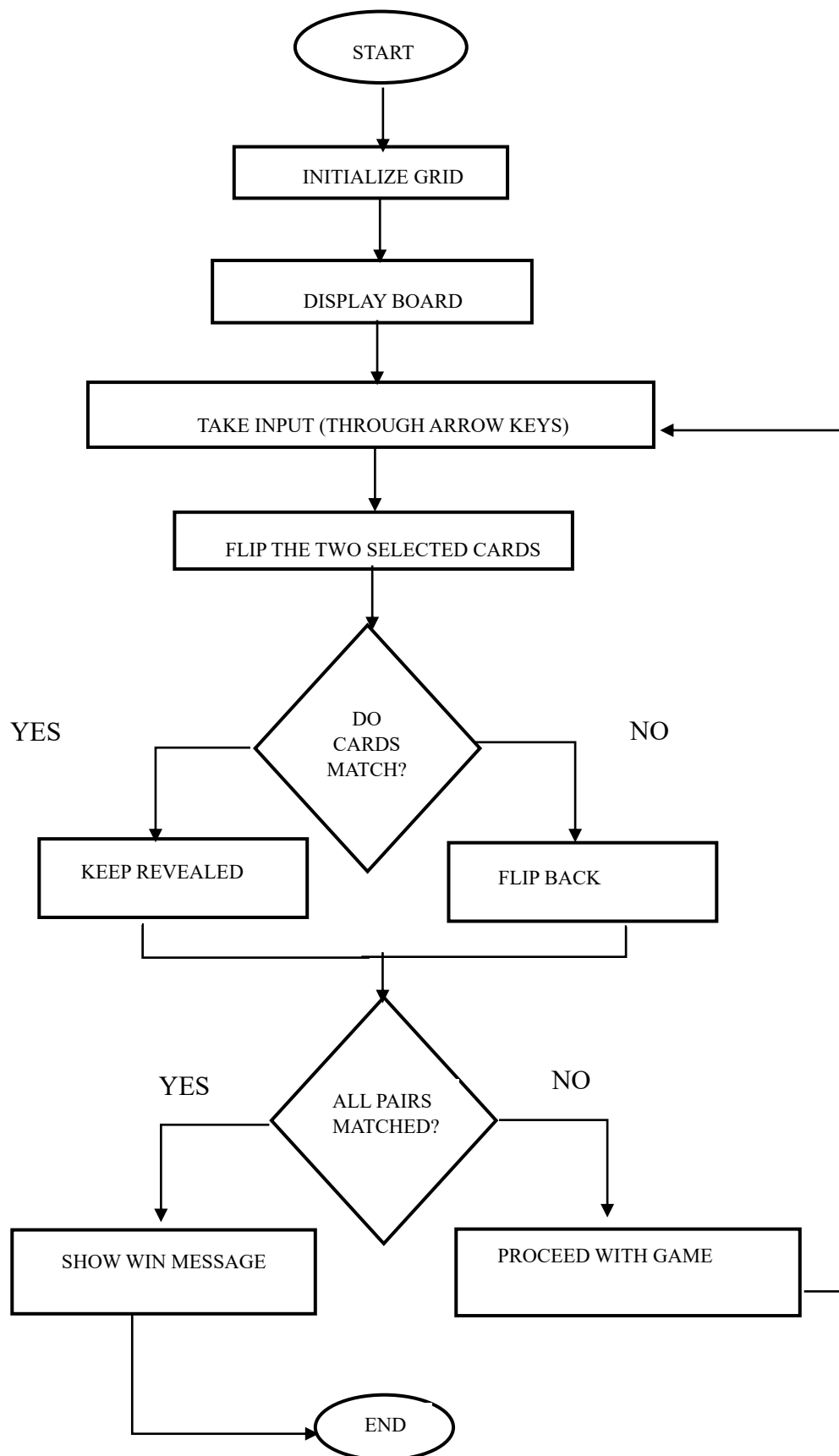
- Turbo C++ Compiler
- DOSBox Emulator (for running Turbo C on modern systems)
- Windows or Linux operating system

▪ FUNCTIONAL & NON-FUNCTIONAL REQUIREMENTS

The functional requirements of the Memory Game specify what the game should do. It shows a 4x4 grid of hidden cards, allowing the player to navigate using arrow keys, and flip cards when selected. Matched pairs stay revealed while unmatched cards are flipped back. The game displays a “YOU WIN!” message when all pairs are successfully matched. The non-functional requirements explain how the game should perform. It should offer a simple and user-friendly keyboard interface for usability, smooth and responsive gameplay for performance, and game must also run on any system with Turbo C and DOSBox for flexibility.

IV. METHODOLOGY

The project was developed using the C programming language with Turbo C graphics. The design follows a step-by-step approach:



Main Components:

START: Begins the game and sets up initial conditions.

INITIALIZE GRID: Randomly arranges pairs of cards on the 4x4 board.

DISPLAY BOARD: Shows the current state of the board with hidden or revealed cards.

TAKE INPUT: Captures player movement and card selection using arrow keys.

FLIP SELECTED CARDS: Reveals the two selected cards.

MATCH CHECK: Determines if the selected cards form a pair.

KEEP REVEALED / FLIP BACK: Keeps matching cards visible or flips unmatched cards back.

ALL PAIRS MATCHED AND CONTINUE: Checks if all pairs are matched and displays the winning message, if not proceeds with the game by taking the input from user.

END: Concludes the game after all pairs are matched.

V. IMPLEMENTATION

The complete source code of the Memory Game demonstrates how the game logic, card matching, and graphics are implemented in C using Turbo C. The program uses arrays to store card values and tracks revealed pairs. This allows the player to interact with the game using keyboard. Running this code displays the 4x4 card grid and handles all gameplay actions, including the winning message.

▪ SOURCE CODE

```
/* Memory Game (4x4) - Arrow-key navigation
```

```
Turbo C compatible C code */
```

```
#include <graphics.h>
```

```
#include <conio.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
#include <dos.h>
```

```
#define SIZE 4
```

```
#define CARDS (SIZE * SIZE)
```

```
int grid[SIZE][SIZE];
```

```
int revealed[SIZE][SIZE];
```

```
int matched[SIZE][SIZE];
```

```
int cursorRow = 0;
```

```
int cursorCol = 0;
```

```
void drawCard(int row, int col, int show);
```

```

void drawBoard(void);
int gameWon(void);

/* draw one card at row,col; show=1 forces face-up */
void drawCard(int row, int col, int show)
{
    int x, y, value;
    char str[4];

    x = 100 + col * 70;
    y = 100 + row * 70;
    value = grid[row][col];

    setcolor(WHITE);
    rectangle(x, y, x + 60, y + 60);

    if (show || revealed[row][col] || matched[row][col]) {
        sprintf(str, "%d", value);
        settxtstyle(DEFAULT_FONT, HORIZ_DIR, 2);
        outtextxy(x + 20, y + 20, str);
    } else {
        setfillstyle(SOLID_FILL, LIGHTBLUE);
        bar(x + 1, y + 1, x + 59, y + 59);
    }

    /* highlight the cursor */
    if (row == cursorRow && col == cursorCol) {
        setcolor(YELLOW);
        rectangle(x - 2, y - 2, x + 62, y + 62);
    }
}

```

```

}

/* draw whole board */
void drawBoard(void)
{
    int i, j;
    cleardevice();
    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            drawCard(i, j, 0);
        }
    }
}

/* check if all matched */
int gameWon(void)
{
    int i, j;
    for (i = 0; i < SIZE; i++) {
        for (j = 0; j < SIZE; j++) {
            if (!matched[i][j]) return 0;
        }
    }
    return 1;
}

int main(void)
{
    int gd, gm;
    int i, j, k, tmp;

```

```

int values[CARDS];
int firstRow, firstCol;
int ch;

gd = DETECT;
initgraph(&gd, &gm, "C:\\TURBOC3\\BGI"); /* change path if needed */

srand((unsigned)time(NULL));

/* prepare pairs (1..CARDS/2) */
for (i = 0; i < CARDS / 2; i++) {
    values[2 * i] = i + 1;
    values[2 * i + 1] = i + 1;
}

/* shuffle */
for (i = 0; i < CARDS; i++) {
    j = rand() % CARDS;
    tmp = values[i];
    values[i] = values[j];
    values[j] = tmp;
}

/* fill grid and reset flags */
k = 0;
for (i = 0; i < SIZE; i++) {
    for (j = 0; j < SIZE; j++) {
        grid[i][j] = values[k++];
        revealed[i][j] = 0;
        matched[i][j] = 0;
    }
}

```

```

    }
}

firstRow = -1;
firstCol = -1;

/* brief on-screen instructions (will be cleared by drawBoard) */
cleardevice();
setcolor(WHITE);
settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
outtextxy(50, 100, "MEMORY GAME (4x4)");
outtextxy(50, 150, "Use ARROW KEYS to choose a card.");
outtextxy(50, 180, "Press ENTER or SPACE to flip a card.");
outtextxy(50, 210, "Match all pairs to win!");
outtextxy(50, 250, "Press ESC anytime to quit.");
outtextxy(50, 300, "Press any key to start...");
getch(); // waits for player

/* main loop */
while (1) {
    drawBoard();

    if (gameWon()) {
        settextstyle(DEFAULT_FONT, HORIZ_DIR, 2);
        outtextxy(150, 30, "ALL PAIRS MATCHED!");
        outtextxy(150, 50, "Press any key to exit");
        getch();
        break;
    }
}

```

```

ch = getch();
if (ch == 0) {
    ch = getch(); /* extended key: arrow keys come as 0 then code */
}

if (ch == 27) { /* ESC to quit */
    break;
}

switch (ch) {
    case 72: /* up */
        if (cursorRow > 0) cursorRow--;
        break;
    case 80: /* down */
        if (cursorRow < SIZE - 1) cursorRow++;
        break;
    case 75: /* left */
        if (cursorCol > 0) cursorCol--;
        break;
    case 77: /* right */
        if (cursorCol < SIZE - 1) cursorCol++;
        break;
    case 13: /* Enter */
    case 32: /* Space */
        if (!matched[cursorRow][cursorCol] && !revealed[cursorRow][cursorCol]) {
            revealed[cursorRow][cursorCol] = 1;
            drawBoard();
            if (firstRow == -1) {
                /* first selection */
                firstRow = cursorRow;
            }
        }
    }
}

```

```

        firstCol = cursorCol;
    } else {
        /* second selection — show briefly, then check */
        delay(800);
        if (grid[firstRow][firstCol] == grid[cursorRow][cursorCol]) {
            matched[firstRow][firstCol] = 1;
            matched[cursorRow][cursorCol] = 1;
        } else {
            revealed[firstRow][firstCol] = 0;
            revealed[cursorRow][cursorCol] = 0;
        }
        firstRow = -1;
        firstCol = -1;
    }
}
break;
default:
    break;
}
}
closegraph();
return 0;
}

```


- THE FOLLOWING SCREENSHOTS ILLUSTRATE THE VARIOUS SCREENS OF THE MEMORY GAME, INCLUDING GAMEPLAY AND RESULTS

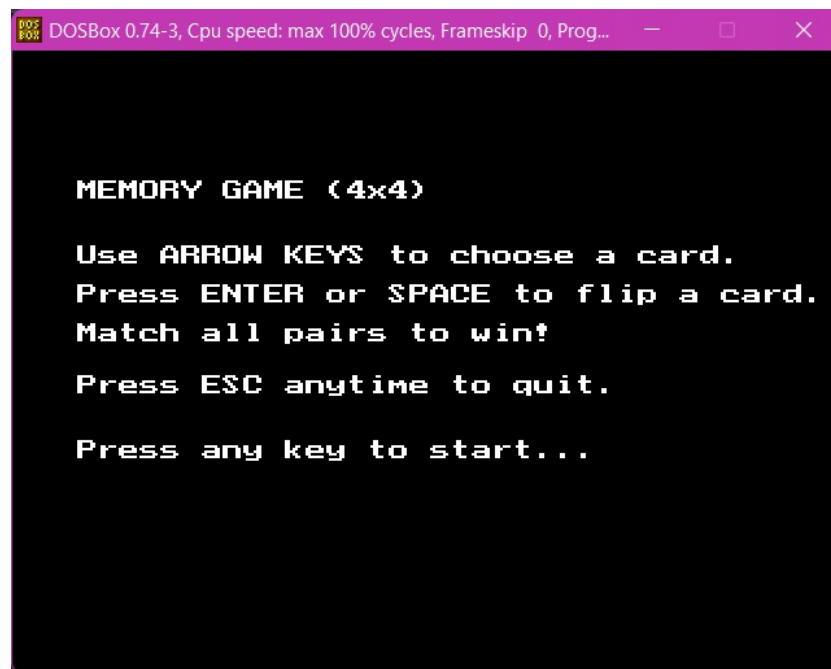


Fig. 1: Instruction screen displayed at the start of the game.

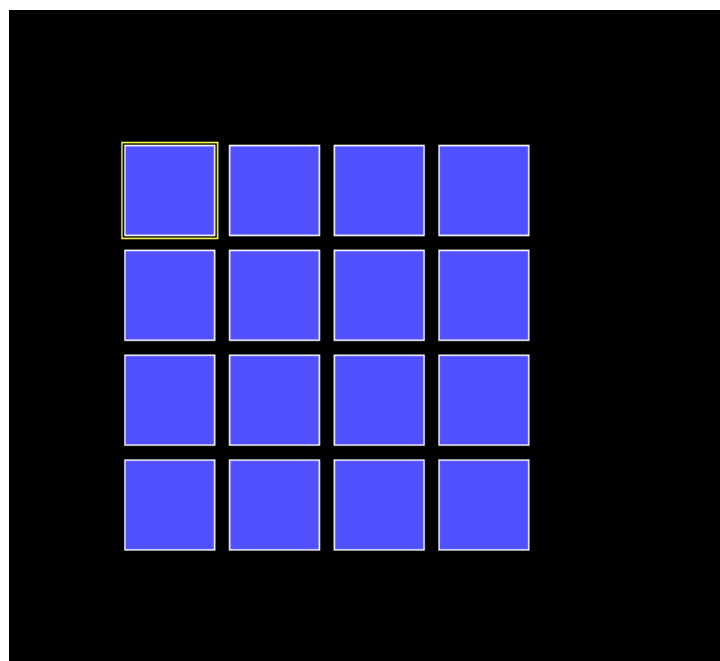


Fig. 2: Gameplay screen with all cards hidden for the player to select.

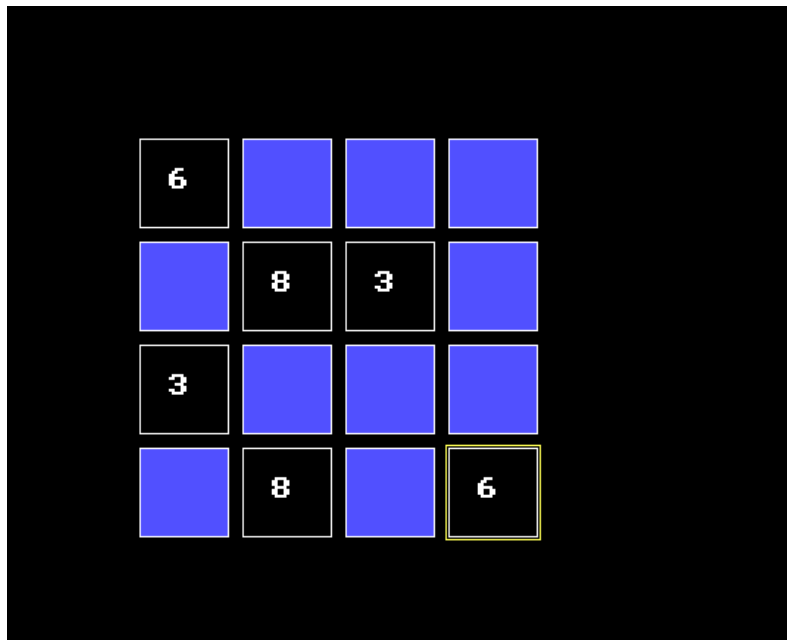


Fig. 3: Gameplay after flipping cards, showing matched pair revealed.

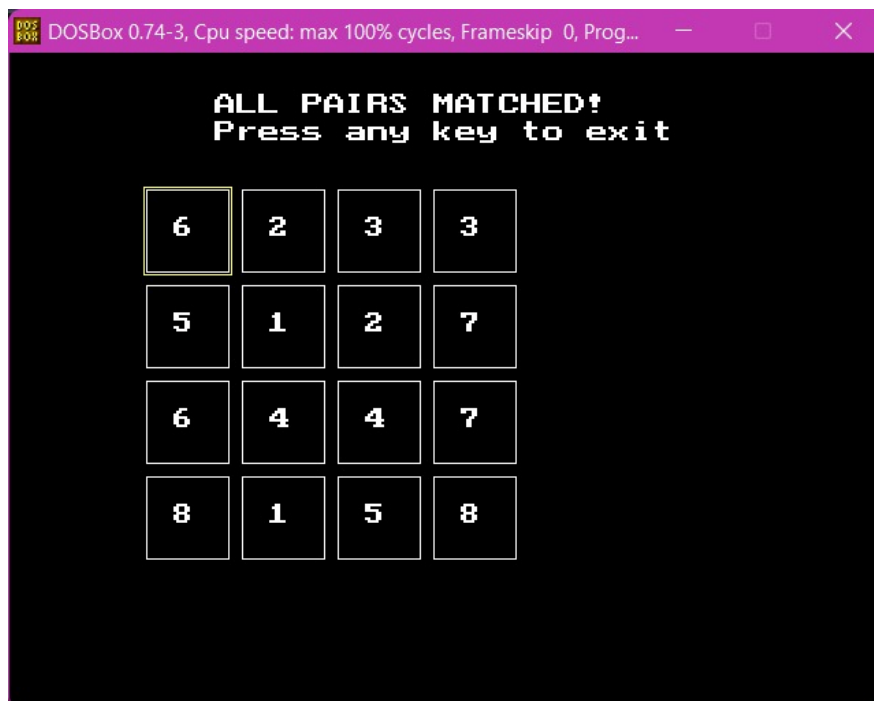


Fig. 4: Winning screen when all pairs are successfully matched.

VI. TESTING

The Memory Game was tested to make sure all functions work as intended. The testing strategy focused on checking gameplay, input handling, match detection, and the winning condition.

▪ TEST CASES

TEST CASE ID	DESCRIPTION	INPUT / ACTION	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC1	Flip two matching cards	Select 2 same numbers	Both cards remain revealed	Works correctly	Pass
TC2	Flip two different cards	Select 2 different numbers	Both cards flip back automatically	Works correctly	Pass
TC3	Cursor navigation	Use arrow keys	Cursor moves correctly around the grid	Works correctly	Pass
TC4	Complete all matches	Play full game	Winning message is displayed	Works correctly	Pass
TC5	Exit game	Press ESC	Game exits safely	Works correctly	Pass

VII. CONCLUSION & FUTURE SCOPE

In this project, I successfully developed a Memory Game of Cards using C programming in the Turbo C environment. The game consists of a 4x4 grid where players flip cards to find matching pairs. It includes features such as randomized card placement, keyboard-based navigation, card flipping, match detection, and a winning message when all pairs are found.

This project allowed me to apply fundamental programming concepts in a practical scenario. I gained hands-on experience in using arrays to store card values, implementing loops and conditional statements to control the game flow, and managing user input for interactive gameplay. Additionally, I learned to use `graphics.h` to create a visual interface, which improved the user experience beyond simple console output.

While the project successfully shows these concepts, it has some limitations. It runs only in Turbo C, so it needs DOSBox on modern systems, and it has a fixed 4x4 grid without adjustable difficulty levels. The visuals are limited to numbers rather than images or animations, and the game doesn't have sound effects or a scoring system. Despite these limitations, there is considerable potential for future improvements. The game can be improved by adding mouse-based selection, implementing different grid sizes and difficulty levels, introducing a timer and scoring system, and including images, animations, and sound effects to make it more interactive and engaging.

Overall, the project not only helped me understand how to combine logic, randomization, and graphics in a single program but also strengthened my problem-solving, debugging, and programming skills. Creating a fully functional, playable game gave me a clear sense of how theoretical concepts in C can be applied to develop interactive software.